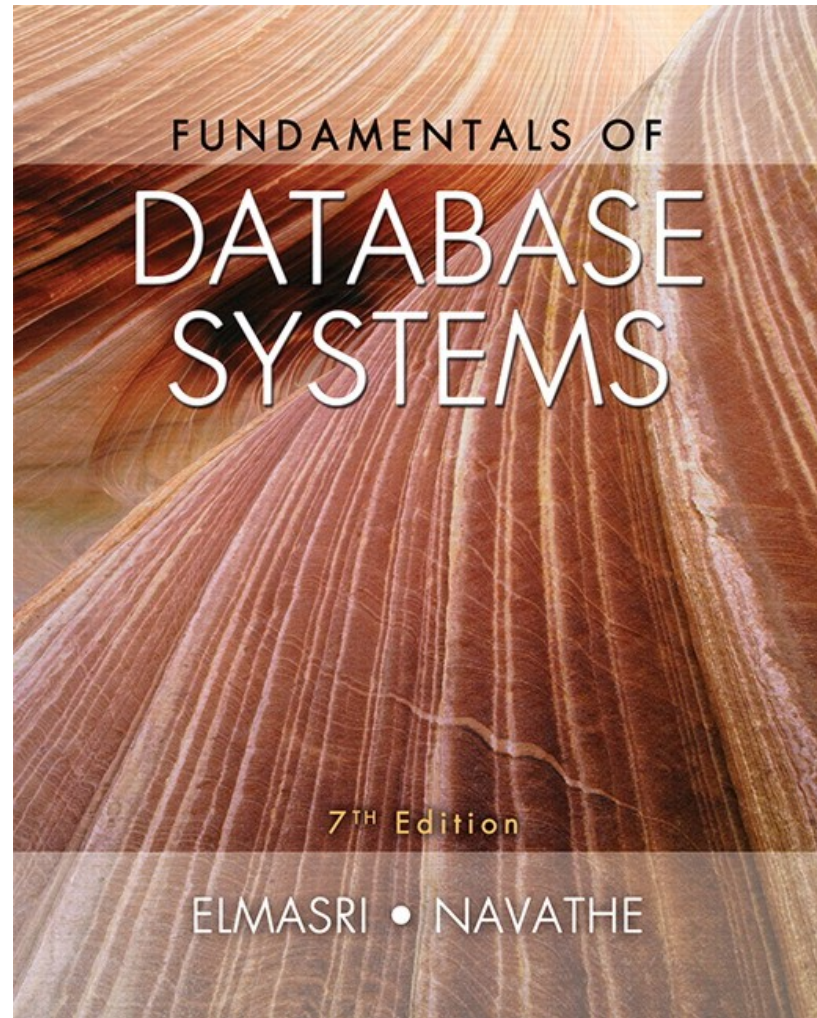




CHAPTER 6

Basic SQL

6.3 Basic Retrieval Queries in SQL

Basic Retrieval Queries in SQL

- `SELECT` statement
 - One basic statement for retrieving information from a database
- SQL allows a table to have two or more tuples that are identical in all their attribute values
 - Unlike relational model (relational model is strictly set-theory based)
 - Multiset or bag behavior
 - Tuple-id may be used as a key

The SELECT-FROM-WHERE Structure of Basic SQL Queries

■ Basic form of the `SELECT` statement:

```
SELECT    <attribute list>  
FROM      <table list>  
WHERE     <condition>;
```

where

- <attribute list> is a list of attribute names whose values are to be retrieved by the query.
- <table list> is a list of the relation names required to process the query.
- <condition> is a conditional (Boolean) expression that identifies the tuples to be retrieved by the query.

The SELECT-FROM-WHERE Structure of Basic SQL Queries (cont'd.)

- Logical comparison operators
 - =, <, <=, >, >=, and <>
- **Projection attributes**
 - Attributes whose values are to be retrieved
- **Selection condition**
 - Boolean condition that must be true for any retrieved tuple. Selection conditions include join conditions (see Ch.8) when multiple relations are involved.

Basic Retrieval Queries

<u>Bdate</u>	<u>Address</u>
1965-01-09	731 Fondren, Houston, TX

<u>Fname</u>	<u>Lname</u>	<u>Address</u>
John	Smith	731 Fondren, Houston, TX
Franklin	Wong	638 Voss, Houston, TX
Ramesh	Narayan	975 Fire Oak, Humble, TX
Joyce	English	5631 Rice, Houston, TX

Query 0. Retrieve the birth date and address of the employee(s) whose name is 'John B. Smith'.

Q0: **SELECT** Bdate, Address
 FROM EMPLOYEE
 WHERE Fname='John' **AND** Minit='B' **AND** Lname='Smith';

Query 1. Retrieve the name and address of all employees who work for the 'Research' department.

Q1: **SELECT** Fname, Lname, Address
 FROM EMPLOYEE, DEPARTMENT
 WHERE Dname='Research' **AND** Dnumber=Dno;

Basic Retrieval Queries (Contd.)

(c)

<u>Pnumber</u>	<u>Dnum</u>	<u>Lname</u>	<u>Address</u>	<u>Bdate</u>
10	4	Wallace	291Berry, Bellaire, TX	1941-06-20
30	4	Wallace	291Berry, Bellaire, TX	1941-06-20

Query 2. For every project located in 'Stafford', list the project number, the controlling department number, and the department manager's last name, address, and birth date.

Q2: **SELECT** Pnumber, Dnum, Lname, Address, Bdate
 FROM PROJECT, DEPARTMENT, EMPLOYEE
 WHERE Dnum=Dnumber **AND** Mgr_ssn=Ssn **AND**
 Plocation='Stafford';

Ambiguous Attribute Names

- Same name can be used for two (or more) attributes in different relations
 - As long as the attributes are in different relations
 - Must **qualify** the attribute name with the relation name to prevent ambiguity

```
Q1A:  SELECT  Fname, EMPLOYEE.Name, Address
        FROM    EMPLOYEE, DEPARTMENT
        WHERE   DEPARTMENT.Name='Research' AND
                DEPARTMENT.Dnumber=EMPLOYEE.Dnumber;
```


Aliasing, and Renaming

- **Aliases or tuple variables**

- Declare alternative relation names E and S to refer to the EMPLOYEE relation twice in a query:

Query 8. For each employee, retrieve the employee's first and last name and the first and last name of his or her immediate supervisor.

- ```
SELECT E.Fname, E.Lname, S.Fname, S.Lname
FROM EMPLOYEE AS E, EMPLOYEE AS S
WHERE E.Super_ssn=S.Ssn;
```

- Recommended practice to abbreviate names and to prefix same or similar attribute from multiple tables.

# Aliasing, Renaming and Tuple Variables (contd.)

- The attribute names can also be renamed

```
EMPLOYEE AS E (Fn, Mi, Ln, Ssn, Bd,
Addr, Sex, Sal, Sssn, Dno)
```

- Note that the relation **EMPLOYEE** now has a variable name **E** which corresponds to a tuple variable
- The “AS” may be dropped in most SQL implementations

# Unspecified WHERE Clause and Use of the Asterisk

- Missing WHERE clause
  - Indicates no condition on tuple selection
- Effect is a CROSS PRODUCT
  - Result is all possible tuple combinations (or the Algebra operation of Cartesian Product– see Ch.8)

Queries 9 and 10. Select all EMPLOYEE Ssns (Q9) and all combinations of EMPLOYEE Ssn and DEPARTMENT Dname (Q10) in the database.

Q9:     SELECT     Ssn  
          FROM     EMPLOYEE;

Q10:    SELECT     Ssn, Dname  
          FROM     EMPLOYEE, DEPARTMENT;

# Unspecified WHERE Clause and Use of the Asterisk (cont'd.)

- Specify an asterisk (\*)
  - Retrieve all the attribute values of the selected tuples
  - The \* can be prefixed by the relation name; e.g., EMPLOYEE \*

**Q1C:**    **SELECT**        \*

**FROM**        EMPLOYEE

**WHERE**       Dno=5;

**Q1D:**    **SELECT**        \*

**FROM**        EMPLOYEE, DEPARTMENT

**WHERE**       Dname='Research' **AND** Dno=Dnumber;

**Q10A:**   **SELECT**        \*

**FROM**        EMPLOYEE, DEPARTMENT;

# Tables as Sets in SQL

- SQL does not automatically eliminate duplicate tuples in query results
- For aggregate operations (See sec 7.1.7) duplicates must be accounted for
- Use the keyword **DISTINCT** in the `SELECT` clause
  - Only distinct tuples should remain in the result

**Query 11.** Retrieve the salary of every employee (Q11) and all distinct salary values (Q11A).

**Q11:**     **SELECT**     **ALL** Salary  
             **FROM**       **EMPLOYEE;**

**Q11A:**    **SELECT**     **DISTINCT** Salary  
             **FROM**       **EMPLOYEE;**

# Tables as Sets in SQL (cont'd.)

- Set operations
  - **UNION, EXCEPT** (difference), **INTERSECT**
  - Corresponding multiset operations: **UNION ALL, EXCEPT ALL, INTERSECT ALL**)
  - Type compatibility is needed for these operations to be valid

**Query 4.** Make a list of all project numbers for projects that involve an employee whose last name is 'Smith', either as a worker or as a manager of the department that controls the project.

```
Q4A: (SELECT DISTINCT Pnumber
 FROM PROJECT, DEPARTMENT, EMPLOYEE
 WHERE Dnum=Dnumber AND Mgr_ssn=Ssn
 AND Lname='Smith')

 UNION
 (SELECT DISTINCT Pnumber
 FROM PROJECT, WORKS_ON, EMPLOYEE
 WHERE Pnumber=Pno AND Essn=Ssn
 AND Lname='Smith');
```

# Substring Pattern Matching and Arithmetic Operators

- **LIKE** comparison operator
  - Used for string **pattern matching**
  - % replaces an arbitrary number of zero or more characters
  - underscore (\_) replaces a single character
  - Examples:
    - **WHERE** Address **LIKE** '%Houston,TX%';
    - **WHERE** Ssn **LIKE** '\_\_ 1\_\_ 8901';
- **BETWEEN** comparison operator
  - Example:
    - **WHERE**(Salary **BETWEEN** 30000 **AND** 40000) **AND** Dno = 5;

# Arithmetic Operations

- Standard arithmetic operators:
  - Addition (+), subtraction (−), multiplication (\*), and division (/) may be included as a part of **SELECT**
- **Query 13.** Show the resulting salaries if every employee working on the 'ProductX' project is given a 10 percent raise.

```
SELECT E.Fname, E.Lname, 1.10 * E.Salary AS Increased_sal
FROM EMPLOYEE AS E, WORKS_ON AS W, PROJECT AS P
WHERE E.Ssn=W.Essn AND W.Pno=P.Pnumber AND
P.Pname='ProductX';
```



# Ordering of Query Results

- Use **ORDER BY** clause
  - Keyword **DESC** to see result in a descending order of values
  - Keyword **ASC** to specify ascending order explicitly (The default)
  - Typically placed at the end of the query

```
ORDER BY D.Dname DESC, E.Lname ASC,
 E.Fname ASC
```

# Basic SQL Retrieval Query Block

```
SELECT <attribute list>
FROM <table list>
[WHERE <condition>]
[ORDER BY <attribute list>];
```

# CHAPTER 7

## More SQL: Complex Queries, Triggers, Views, and Schema Modification

# Chapter 7 Outline

- 7.1 More Complex SQL Retrieval Queries
- 7.3 Views (Virtual Tables) in SQL

# More Complex SQL Retrieval Queries

- Additional features allow users to specify more complex retrievals from database:
  - Nested queries, joined tables, and outer joins (in the FROM clause), aggregate functions, and grouping

## 7.1.1 Comparisons Involving NULL and Three-Valued Logic

- Meanings of NULL
  - **Unknown value**
  - **Unavailable or withheld value**
  - **Not applicable attribute**
- Each individual NULL value considered to be different from every other NULL value
- SQL uses a three-valued logic:
  - TRUE, FALSE, and UNKNOWN (like Maybe)
- **NULL = NULL comparison is avoided**

# Comparisons Involving NULL and Three-Valued Logic (cont'd.)

# Comparisons Involving NULL and Three-Valued Logic (cont'd.)

- SQL allows queries that check whether an attribute value is NULL
  - IS or IS NOT NULL

**Query 18.** Retrieve the names of all employees who do not have supervisors.

```
Q18: SELECT Fname, Lname
 FROM EMPLOYEE
 WHERE Super_ssn IS NULL;
```



## 7.1.2 Nested Queries, Tuples, and Set/Multiset Comparisons

- **Nested queries**
  - Complete select-from-where blocks within WHERE clause of another query
  - Some queries require that existing values in the database be fetched and then used in a comparison condition
  - **Outer query and nested subqueries**

# Nested Queries (cont'd.)

- Comparison operator `IN`
  - Compares value  $v$  with a set (or multiset) of values  $V$
  - Evaluates to `TRUE` if  $v$  is one of the elements in  $V$
  - If the results is a single value we can use `=`

```
Q4A: SELECT DISTINCT Pnumber
 FROM PROJECT
 WHERE Pnumber IN
 (SELECT Pnumber
 FROM PROJECT, DEPARTMENT, EMPLOYEE
 WHERE Dnum=Dnumber AND
 Mgr_ssn=Ssn AND Lname='Smith')
 OR
 Pnumber IN
 (SELECT Pno
 FROM WORKS_ON, EMPLOYEE
 WHERE Essn=Ssn AND Lname='Smith');
```

# Nested Queries (cont'd.)

- Use tuples of values in comparisons
  - Place them within parentheses

```
SELECT DISTINCT Essn
FROM WORKS_ON
WHERE (Pno, Hours) IN (SELECT Pno, Hours
 FROM WORKS_ON
 WHERE Essn='123456789');
```

# IN Operator

---

```
SELECT column_name(s)
FROM table_name
WHERE column_name IN (value1, value2, ...);
```

```
SELECT * FROM Customers
WHERE Country IN ('Germany', 'France', 'UK');
```

---

```
SELECT column_name(s)
FROM table_name
WHERE column_name IN (SELECT STATEMENT);
```

---

```
SELECT * FROM Customers
WHERE Country IN (SELECT Country FROM Suppliers);
```

# Nested Queries (cont'd.)

- Use other comparison operators to compare a single value  $v$ 
  - $=$  ANY (or  $=$  SOME) operator
    - Returns TRUE if the value  $v$  is equal to some value in the set  $V$  and is hence equivalent to IN
  - Other operators that can be combined with ANY (or SOME):  $>$ ,  $>=$ ,  $<$ ,  $<=$ , and  $<>$
  - ALL: value must exceed all values from nested query

```
SELECT Lname, Fname
FROM EMPLOYEE
WHERE Salary > ALL (SELECT Salary
 FROM EMPLOYEE
 WHERE Dno=5);
```

# Nested Queries (cont'd.)

- Avoid potential errors and ambiguities
  - Create tuple variables (aliases) for all tables referenced in SQL query
  - The rule is that a reference to an unqualified attribute refers to the relation declared in the innermost nested query

**Query 16.** Retrieve the name of each employee who has a dependent with the same first name and is the same sex as the employee.

```
Q16: SELECT E.Fname, E.Lname
 FROM EMPLOYEE AS E
 WHERE E.Ssn IN (SELECT Essn
 FROM DEPENDENT AS D
 WHERE E.Fname=D.Dependent_name
 AND E.Sex=D.Sex);
```

# Nested Queries (cont'd.)

- **Queries that are nested using the = or IN comparison operator** can be collapsed into one single block: E.g., Q16 can be written as:

```
Q16A: SELECT E.Fname, E.Lname
 FROM EMPLOYEE AS E, DEPENDENT AS D
 WHERE E.Ssn = D.Essn AND E.Sex = D.Sex
 AND E.Fname = D.Dependent_name;
```

# Correlated Nested Queries 7.1.3

- **Correlated nested query**
  - Whenever a condition in the WHERE clause of a nested query references some attribute of a relation declared in the outer query
  - Evaluated once for each tuple in the outer query

```
SELECT E.Fname, E.Lname
FROM EMPLOYEE AS E
WHERE E.Ssn IN (SELECT Essn
 FROM DEPENDENT AS D
 WHERE E.Fname=D.Dependent_name
 AND E.Sex=D.Sex);
```



## 7.1.4 The EXISTS and UNIQUE Functions in SQL for correlating queries

- **EXISTS function**
  - Check whether the result of a correlated nested query is empty or not. They are Boolean functions that return a TRUE or FALSE result.
- **EXISTS and NOT EXISTS**
  - Typically used in conjunction with a correlated nested query
- **SQL function UNIQUE (Q)**
  - Returns TRUE if there are no duplicate tuples in the result of query Q

# USE of EXISTS

|              |               |                        |                                         |
|--------------|---------------|------------------------|-----------------------------------------|
| <b>Q16B:</b> | <b>SELECT</b> | E.Fname, E.Lname       |                                         |
|              | <b>FROM</b>   | EMPLOYEE <b>AS</b> E   |                                         |
|              | <b>WHERE</b>  | <b>EXISTS ( SELECT</b> | *                                       |
|              |               | <b>FROM</b>            | DEPENDENT <b>AS</b> D                   |
|              |               | <b>WHERE</b>           | E.Ssn = D.Essn <b>AND</b> E.Sex = D.Sex |
|              |               |                        | <b>AND</b> E.Fname = D.Dependent_name); |

# USE of EXISTS

**Query 7.** List the names of managers who have at least one dependent.

```
Q7: SELECT Fname, Lname
 FROM EMPLOYEE
 WHERE EXISTS (SELECT *
 FROM DEPENDENT
 WHERE Ssn = Essn)
 AND
 EXISTS (SELECT *
 FROM DEPARTMENT
 WHERE Ssn = Mgr_ssn);
```

## 7.1.5 Explicit Sets and Renaming of Attributes in SQL

- Can use explicit set of values in WHERE clause

**Query 17.** Retrieve the Social Security numbers of all employees who work on project numbers 1, 2, or 3.

```
Q17: SELECT DISTINCT Essn
 FROM WORKS_ON
 WHERE Pno IN (1, 2, 3);
```

- Use qualifier AS followed by desired new name
  - Rename any attribute that appears in the result of a query

```
SELECT E.Lname AS Employee_name, S.Lname AS Supervisor_name
FROM EMPLOYEE AS E, EMPLOYEE AS S
WHERE E.Super_ssn = S.Ssn;
```

## 7.1.6 Specifying Joined Tables in the FROM Clause of SQL

- **Joined table**

- Permits users to specify a table resulting from a join operation in the FROM clause of a query

- **The FROM clause in Q1A**

- Contains a single joined table. JOIN may also be called INNER JOIN

```
Q1A: SELECT Fname, Lname, Address
 FROM (EMPLOYEE JOIN DEPARTMENT ON Dno=Dnumber)
 WHERE Dname='Research';
```

# Different Types of JOINed Tables in SQL

- Specify different types of join
  - NATURAL JOIN
  - Various types of OUTER JOIN (LEFT, RIGHT, FULL)
- NATURAL JOIN on two relations R and S
  - No join condition specified
  - Is equivalent to an implicit EQUIJOIN condition for each pair of attributes with same name from R and S

# NATURAL JOIN

- Rename attributes of one relation so it can be joined with another using NATURAL JOIN:

```
SELECT Fname, Lname, Address
FROM (EMPLOYEE NATURAL JOIN
 (DEPARTMENT AS DEPT (Dname, Dno, Mssn, Msdate)))
WHERE Dname = 'Research';
```

- The above works with  $EMPLOYEE.Dno = DEPT.Dno$  as an implicit join condition

# Example: LEFT OUTER JOIN

```
SELECT E.Lname AS Employee_name,
 S.Lname AS Supervisor_name
FROM (EMPLOYEE AS E LEFT OUTER JOIN EMPLOYEE AS S
 ON E.Super_ssn = S.Ssn);
```

## ALTERNATE SYNTAX:

```
SELECT E.Lname, S.Lname
FROM EMPLOYEE E, EMPLOYEE S
WHERE E.Super_ssn + = S.Ssn;
```



# INNER and OUTER Joins

- **INNER JOIN (versus OUTER JOIN)**
  - Default type of join in a joined table
  - Tuple is included in the result only if a matching tuple exists in the other relation
- **LEFT OUTER JOIN**
  - Every tuple in left table must appear in result
  - If no matching tuple
    - Padded with NULL values for attributes of right table
- **RIGHT OUTER JOIN**
  - Every tuple in right table must appear in result
  - If no matching tuple
    - Padded with NULL values for attributes of left table

# Multiway JOIN in the FROM clause

- FULL OUTER JOIN – combines result if LEFT and RIGHT OUTER JOIN
- Can nest JOIN specifications for a multiway join:

```
SELECT Pnumber, Dnum, Lname, Address, Bdate
FROM ((PROJECT JOIN DEPARTMENT ON Dnum = Dnumber)
 JOIN EMPLOYEE ON Mgr_ssn = Ssn)
WHERE Plocation = 'Stafford';
```

## 7.1.7 Aggregate Functions in SQL

- Used to summarize information from multiple tuples into a single-tuple summary
- Built-in aggregate functions
  - **COUNT**, **SUM**, **MAX**, **MIN**, and **AVG**
- **Grouping**
  - Create subgroups of tuples before summarizing
- To select entire groups, **HAVING** clause is used
- Aggregate functions can be used in the **SELECT** clause or in a **HAVING** clause

# Renaming Results of Aggregation

- Following query returns a single row of computed values from EMPLOYEE table:

```
SELECT SUM (Salary), MAX (Salary), MIN (Salary), AVG (Salary)
FROM EMPLOYEE;
```

- The result can be presented with new names:

```
SELECT SUM (Salary) AS Total_Sal, MAX (Salary) AS Highest_Sal,
 MIN (Salary) AS Lowest_Sal, AVG (Salary) AS Average_Sal
FROM EMPLOYEE;
```

# Aggregate Functions in SQL (cont'd.)

- NULL values are discarded when aggregate functions are applied to a particular column

**Query 20.** Find the sum of the salaries of all employees of the 'Research' department, as well as the maximum salary, the minimum salary, and the average salary in this department.

```
Q20: SELECT SUM (Salary), MAX (Salary), MIN (Salary), AVG (Salary)
 FROM (EMPLOYEE JOIN DEPARTMENT ON Dno=Dnumber)
 WHERE Dname='Research';
```

**Queries 21 and 22.** Retrieve the total number of employees in the company (Q21) and the number of employees in the 'Research' department (Q22).

```
Q21: SELECT COUNT (*)
 FROM EMPLOYEE;
```

```
Q22: SELECT COUNT (*)
 FROM EMPLOYEE, DEPARTMENT
 WHERE DNO=DNUMBER AND DNAME='Research';
```

# Aggregate Functions on Booleans

- SOME and ALL may be applied as functions on Boolean Values.
- SOME returns true if at least one element in the collection is TRUE (similar to OR)
- ALL returns true if all of the elements in the collection are TRUE (similar to AND)

## 7.1.8 Grouping: The GROUP BY Clause

- **Partition** relation into subsets of tuples
  - Based on **grouping attribute(s)**
  - Apply function to each such group independently
- **GROUP BY** clause
  - Specifies grouping attributes
- **COUNT (\*)** counts the number of rows in the group

# Examples of GROUP BY

- The grouping attribute must appear in the SELECT clause:

```
SELECT Dno, COUNT (*), AVG (Salary)
FROM EMPLOYEE
GROUP BY Dno;
```

- If the grouping attribute has NULL as a possible value, then a separate group is created for the null value (e.g., null Dno in the above query)
- GROUP BY may be applied to the result of a JOIN:

```
SELECT Pnumber, Pname, COUNT (*)
FROM PROJECT, WORKS_ON
WHERE Pnumber = Pno
GROUP BY Pnumber, Pname;
```



# Grouping: The GROUP BY and HAVING Clauses (cont'd.)

- **HAVING** clause
  - Provides a condition to select or reject an entire group:

**Query 26.** For each project *on which more than two employees work*, retrieve the project number, the project name, and the number of employees who work on the project.

**Q26:**

|                 |                                  |
|-----------------|----------------------------------|
| <b>SELECT</b>   | Pnumber, Pname, <b>COUNT</b> (*) |
| <b>FROM</b>     | PROJECT, WORKS_ON                |
| <b>WHERE</b>    | Pnumber = Pno                    |
| <b>GROUP BY</b> | Pnumber, Pname                   |
| <b>HAVING</b>   | <b>COUNT</b> (*) > 2;            |

# Combining the WHERE and the HAVING Clause

- Consider the query: we want to count the *total* number of employees whose salaries exceed \$40,000 in each department, but only for departments where more than five employees work.
- **INCORRECT QUERY:**

```
SELECT Dno, COUNT (*)
FROM EMPLOYEE
WHERE Salary>40000
GROUP BY Dno
HAVING COUNT (*) > 5;
```

# Combining the WHERE and the HAVING Clause (continued)

## Correct Specification of the Query:

- Note: the WHERE clause applies tuple by tuple whereas HAVING applies to entire group of tuples

**Query 28.** For each department that has more than five employees, retrieve the department number and the number of its employees who are making more than \$40,000.

```
Q28: SELECT Dnumber, COUNT (*)
 FROM DEPARTMENT, EMPLOYEE
 WHERE Dnumber=Dno AND Salary>40000 AND
 (SELECT Dno
 FROM EMPLOYEE
 GROUP BY Dno
 HAVING COUNT (*) > 5)
```

## 7.1.9 Use of WITH

- The WITH clause allows a user to define a table that will only be used in a particular query (not available in all SQL implementations)
- Used for convenience to create a temporary “View” and use that immediately in a query
- Allows a more straightforward way of looking a step-by-step query

# Example of WITH

- See an alternate approach to doing Q28:

```
WITH BIGDEPTS (Dno) AS
 (SELECT Dno|
 FROM EMPLOYEE
 GROUP BY Dno
 HAVING COUNT (*) > 5)

SELECT Dno, COUNT (*)
FROM EMPLOYEE
WHERE Salary>40000 AND Dno IN BIGDEPTS
GROUP BY Dno;
```

# Use of CASE

- SQL also has a CASE construct
- Used when a value can be different based on certain conditions.
- Can be used in any part of an SQL query where a value is expected
- Applicable when querying, inserting or updating tuples

# EXAMPLE of use of CASE

- The following example shows that employees are receiving different raises in different departments (A variation of the update U6)

```
U6': UPDATE EMPLOYEE
 SET Salary =
 CASE
 WHEN Dno = 5 THEN Salary + 2000
 WHEN Dno = 4 THEN Salary + 1500
 WHEN Dno = 1 THEN Salary + 3000
 ELSE
 Salary + 0 ;
```

## 7.1.10 Recursive Queries in SQL

- An example of a **recursive relationship** between tuples of the same type is the relationship between an employee and a supervisor.
- This relationship is described by the foreign key Super\_ssn of the EMPLOYEE relation
- An example of a **recursive operation** is to retrieve all supervisees of a supervisory employee  $e$  at all levels—that is, all employees  $e'$  directly supervised by  $e$ , all employees  $e''$  directly supervised by each employee  $e'$ , all employees  $e'''$  directly supervised by each employee  $e''$ , and so on. Thus the CEO would have each employee in the company as a supervisee in the resulting table. Example shows such table SUP\_EMP with 2 columns (Supervisor, Supervisee(any level)):



# An EXAMPLE of RECURSIVE Query

- The query starts with an empty SUP\_EMP and successively builds SUP\_EMP table by computing immediate supervisees first, then second level supervisees, etc. until a **fixed point** is reached and no more supervisees can be added

```
Q29: WITH RECURSIVE SUP_EMP (SupSsn, EmpSsn) AS
 (SELECT SupervisorSsn, Ssn
 FROM EMPLOYEE
 UNION
 SELECT E.Ssn, S.SupSsn
 FROM EMPLOYEE AS E, SUP_EMP AS S
 WHERE E.SupervisorSsn = S.EmpSsn)

 SELECT*
 FROM SUP_EMP;
```

# EXPANDED Block Structure of SQL Queries

```
SELECT <attribute and function list>
FROM <table list>
[WHERE <condition>]
[GROUP BY <grouping attribute(s)>]
[HAVING <group condition>]
[ORDER BY <attribute list>];
```

## 7.3 Views (Virtual Tables) in SQL

- Concept of a view in SQL
  - Single table derived from other tables called the **defining tables**
  - Considered to be a virtual table that is not necessarily populated

# Specification of Views in SQL

## ■ **CREATE VIEW** command

- Give table name, list of attribute names, and a query to specify the contents of the view
- In V1, attributes retain the names from base tables. In V2, attributes are assigned names

```
V1: CREATE VIEW WORKS_ON1
 AS SELECT
 Fname, Lname, Pname, Hours
 FROM EMPLOYEE, PROJECT, WORKS_ON
 WHERE Ssn = Essn AND Pno = Pnumber;
```

```
V2: CREATE VIEW DEPT_INFO(Dept_name, No_of_emps, Total_sal)
 AS SELECT
 Dname, COUNT (*), SUM (Salary)
 FROM DEPARTMENT, EMPLOYEE
 WHERE Dnumber = Dno
 GROUP BY Dname;
```

# Specification of Views in SQL (cont'd.)

- Once a View is defined, SQL queries can use the View relation in the FROM clause
- View is always up-to-date
  - Responsibility of the DBMS and not the user
- **DROP VIEW** command
  - Dispose of a view

# View Implementation, View Update, and Inline Views

- Complex problem of efficiently implementing a view for querying
- **Strategy1: Query modification** approach
  - Compute the view as and when needed. Do not store permanently
  - Modify view query into a query on underlying base tables
  - Disadvantage: inefficient for views defined via complex queries that are time-consuming to execute

# View Materialization

- **Strategy 2: View materialization**
  - Physically create a temporary view table when the view is first queried
  - Keep that table on the assumption that other queries on the view will follow
  - Requires efficient strategy for automatically updating the view table when the base tables are updated
- **Incremental update strategy for materialized views**
  - DBMS determines what new tuples must be inserted, deleted, or modified in a materialized view table

# View Materialization (contd.)

- Multiple ways to handle materialization:
  - **immediate update** strategy updates a view as soon as the base tables are changed
  - **lazy update** strategy updates the view when needed by a view query
  - **periodic update** strategy updates the view periodically (in the latter strategy, a view query may get a result that is not up-to-date). This is commonly used in Banks, Retail store operations, etc.



# View Update

- Update on a view defined on a single table without any aggregate functions
  - Can be mapped to an update on underlying base table-possible if the primary key is preserved in the view
- Update not permitted on aggregate views. E.g., cannot be processed because Total\_sal is a computed value in the view definition

|             |               |                     |
|-------------|---------------|---------------------|
| <b>UV2:</b> | <b>UPDATE</b> | DEPT_INFO           |
|             | <b>SET</b>    | Total_sal = 100000  |
|             | <b>WHERE</b>  | Dname = 'Research'; |

# View Update and Inline Views

- View involving joins
  - Often not possible for DBMS to determine which of the updates is intended
- Clause **WITH CHECK OPTION**
  - Must be added at the end of the view definition if a view is to be updated to make sure that tuples being updated stay in the view
- **In-line view**
  - Defined in the **FROM** clause of an SQL query (e.g., we saw its used in the **WITH** example)

# Views as authorization mechanism

- SQL query authorization statements (GRANT and REVOKE) are described in detail in Chapter 30
- Views can be used to hide certain attributes or tuples from unauthorized users
- E.g., For a user who is only allowed to see employee information for those who work for department 5, he may only access the view

```
CREATE VIEW DEPT5EMP AS
SELECT *
FROM EMPLOYEE
WHERE Dno = 5;
```

# Table 7.2 Summary of SQL Syntax

**Table 7.2** Summary of SQL Syntax

---

```
CREATE TABLE <table name> (<column name> <column type> [<attribute constraint>]
 { , <column name> <column type> [<attribute constraint>] }
 [<table constraint> { , <table constraint> }])
```

---

```
DROP TABLE <table name>
ALTER TABLE <table name> ADD <column name> <column type>
```

---

```
SELECT [DISTINCT] <attribute list>
FROM (<table name> { <alias> } | <joined table>) { , (<table name> { <alias> } | <joined table>) }
[WHERE <condition>]
[GROUP BY <grouping attributes> [HAVING <group selection condition>]]
[ORDER BY <column name> [<order>] { , <column name> [<order>] }]
```

---

```
<attribute list> ::= (* | (<column name> | <function> (([DISTINCT] <column name> | *)))
 { , (<column name> | <function> (([DISTINCT] <column name> | *))) }))
```

---

```
<grouping attributes> ::= <column name> { , <column name> }
```

---

```
<order> ::= (ASC | DESC)
```

---

```
INSERT INTO <table name> [(<column name> { , <column name> })]
(VALUES (<constant value> , { <constant value> }) { , (<constant value> { , <constant value> }) }
| <select statement>)
```

---

*continued on next slide*

# Table 7.2 (continued)

## Summary of SQL Syntax

**Table 7.2** Summary of SQL Syntax

---

DELETE FROM <table name>

[ WHERE <selection condition> ]

---

UPDATE <table name>

SET <column name> = <value expression> { , <column name> = <value expression> }

[ WHERE <selection condition> ]

---

CREATE [ UNIQUE] INDEX <index name>

ON <table name> ( <column name> [ <order> ] { , <column name> [ <order> ] } )

[ CLUSTER ]

---

DROP INDEX <index name>

---

CREATE VIEW <view name> [ ( <column name> { , <column name> } ) ]

AS <select statement>

---

DROP VIEW <view name>

---

NOTE: The commands for creating and dropping indexes are not part of standard SQL.

# Summary

- Complex SQL:
  - Nested queries, joined tables (in the FROM clause), outer joins, aggregate functions, grouping
- CREATE VIEW statement and materialization strategies